

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	https://github.com/venkatasai-eng/MLA0403-Deep-learning

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA04

SLOT : B

Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%

Duration: 90 minutes

Assessment Tool Weightage: 30%

Mark : 25

Code of Conduct:

I _Guggilla Venkatasai(192425388)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: G.Venkatasai

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature :

Total Marks:

1. Question 1 - Design an Autoencoder-Based Feature Learning System

1.1 Problem definition

A manufacturing company wants to detect defective products from high-dimensional sensor measurements. The important challenge is that a raw sensor vector may contain hundreds of correlated readings, redundant channels and operating-condition variations. The design therefore learns a compact representation from predominantly normal products and uses reconstruction behavior as a signal for abnormality.

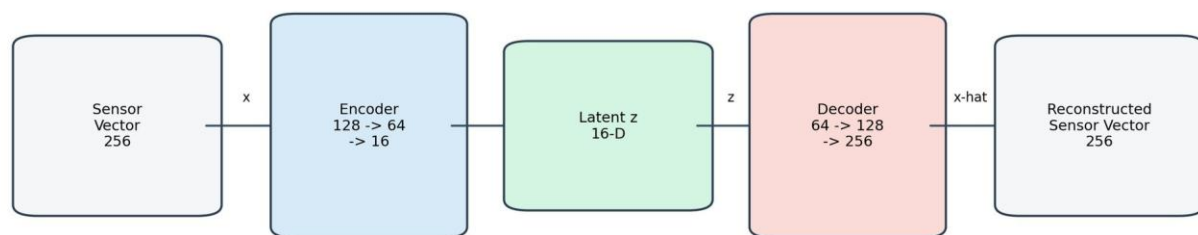
1.2 Requirements, inputs, outputs and constraints

Element	design
Input	256 normalized sensor features for one product or one fixed time window.
Output A	16-dimensional latent vector z for downstream feature learning.
Output B	256-dimensional reconstruction $\hat{x}$ used to compute reconstruction error.
Target use	Defect screening / anomaly detection before manual inspection.
Main constraint	Latent dimension must be substantially smaller than input dimension.
Training data	Preferably mostly trusted normal samples; labelled defects can be reserved for threshold tuning and final evaluation.

1.3 Proposed architecture

Stage	Layer	Units / operation	Activation
Input	Sensor vector	256	-
Encoder 1	Dense	128	ReLU
Encoder 2	Dense	64	ReLU
Bottleneck	Dense	16	Linear / identity
Decoder 1	Dense	64	ReLU
Decoder 2	Dense	128	ReLU
Output	Dense	256	Linear

Q1 - Undercomplete Autoencoder for Defect Detection



Objective: minimize mean squared reconstruction error

Dimensionality reduction: 256 features -> 16 latent features

Figure 2. Undercomplete autoencoder architecture for compact sensor representation.

1.4 Mathematical objective

For a batch of normalized sensor vectors x , the reconstruction objective is:

Reconstruction loss: $L_{\text{rec}} = (1/N) * \sum_i ||x_i - \hat{x}_i||_2^2$. Here $\hat{x}$ is the decoder output and N is the number of training samples.

Adam optimization can minimize this loss over mini-batches. Early stopping based on validation reconstruction loss helps prevent unnecessary training once the representation has stabilized.

1.5 Why dimensionality reduction produces useful features

1. Normalization places sensor channels on comparable scales so that a large-valued feature does not dominate the reconstruction objective.
2. The encoder transforms 256 measurements into a 16-dimensional code that must preserve the information needed for reconstruction.
3. The decoder expands the code back to 256 values, forcing the latent vector to retain correlations and recurring operating patterns.
4. The latent representation can then be used as input to a separate defect classifier or anomaly detector instead of feeding all raw features to the downstream system.

5. 1.6 Defect-detection workflow

1. Train the autoencoder on trusted normal samples.
2. Compute reconstruction error for validation normal samples and known defects.
3. Choose an operating threshold using validation data rather than the test set.
4. For a new product, reconstruct its sensor vector and calculate the reconstruction error.
5. Flag the product as suspicious when the error exceeds the chosen threshold; forward the latent code to a diagnostic model if classification is required.

1.7 Evaluation and expected outcomes

Metric	Purpose	Desired interpretation
MSE / MAE	Measures reconstruction fidelity.	Lower is generally better on representative normal data.
Precision / Recall / F1	Measures defect-screening usefulness.	High recall is valuable when missing a defect is costly.
ROC-AUC / PR-AUC	Evaluates ranking across thresholds.	PR-AUC is informative when defects are rare.
Latent-space visualization	Inspects learned feature organization.	Normal and defective cases may show distinct structure if the representation is informative.

2. Question 2 - Design a Robust Regularized Autoencoder

2.1 Problem definition

Healthcare records can contain noise, missingness, redundant measurements and patient-specific fluctuations. A plain autoencoder can memorize nuisance variation, so the proposed model adds explicit regularization and optional corruption so that the representation captures patterns that remain useful for downstream classification.

2.2 Proposed architecture

Stage	Layer	Units	Activation / operation
Input	Patient feature vector	256	Standardized continuous input
Encoder 1	Dense	128	ReLU
Latent	Dense	32	ReLU or linear code
Decoder 1	Dense	128	ReLU
Output	Dense	256	Linear

Q2 - Robust Regularized Autoencoder

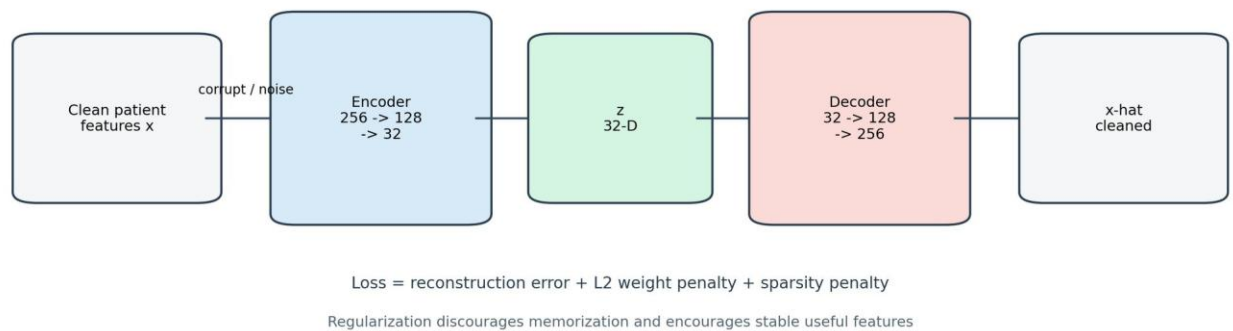


Figure 3. Regularized autoencoder with corruption, bottleneck and explicit regularization.

2.3 Regularization strategy

The design combines two forms of regularization: an L2 weight penalty and a sparsity penalty on latent activations. An optional denoising step masks or perturbs a subset of features so that the model learns to reconstruct a clean record from a corrupted input.

Training objective: $L_{\text{total}} = L_{\text{rec}} + \lambda \|W\|_2^2 + \beta L_{\text{sparse}}$. The reconstruction term preserves useful information; the L2 term discourages excessively large weights; the sparsity term encourages a compact code.

2.4 Training mechanism

1. Standardize continuous variables and handle missing values with a documented preprocessing rule.
2. Create a corrupted version of each training example by masking or perturbing selected features.
3. Encode the corrupted input into a 32-dimensional representation.

4. Decode the representation toward the clean target record.
5. Backpropagate the combined reconstruction and regularization objective.
6. Select the model using validation reconstruction and downstream classification performance rather than training loss alone.

2.5 How it reduces overfitting

- L2 weight decay limits overly complex mappings and reduces dependence on individual training records.
- Sparsity encourages a small number of active latent features for each patient, making the code more compact.
- Input corruption prevents the network from succeeding by simple feature copying and encourages recovery of stable patterns.
- The learned embedding can be frozen and supplied to a lightweight classifier, reducing the classifier input space from 256 raw variables to 32 learned features.

2.6 Evaluation plan

Measure	What it tests
Reconstruction MSE	Whether patient information is retained sufficiently.
Latent sparsity	Whether the representation is compact rather than uniformly active.
Validation F1 / ROC-AUC	Whether the embedding improves downstream classification.
Noise robustness test	Whether performance remains stable after realistic feature corruption.
Calibration / error analysis	Whether the downstream classifier behaves consistently across relevant subgroups or operating conditions.

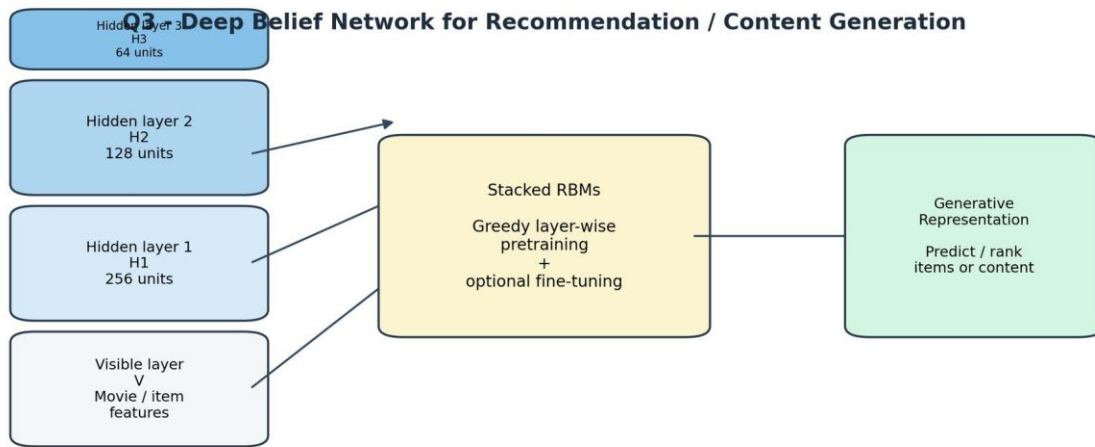
3. Question 3 - Design a Deep Generative Model using a Deep Belief Network

3.1 Application context

A recommendation or content-generation system needs to learn complex relationships among users, items and observed interactions. A Deep Belief Network (DBN) is proposed because stacked Restricted Boltzmann Machines (RBMs) provide layer-wise unsupervised representation learning before task-specific fine-tuning.

3.2 Architecture

Layer	Size	Role
Visible layer V	512	Binary or normalized interaction/content features.
Hidden layer H1	256	Learns first-order co-occurrence structure.
Hidden layer H2	128	Combines lower-level patterns into richer latent features.
Hidden layer H3	64	Forms a compact hierarchical representation.



Choice: DBN gives hierarchical features and can model complex distributions efficiently.

Figure 4. Hierarchical DBN design built from stacked RBMs.

3.3 Learning process

1. Pretrain RBM-1 using the original visible data; learn parameters connecting V and H1.
2. Compute hidden activations from RBM-1 and use them as the training representation for RBM-2.
3. Repeat layer-wise pretraining for H2 and H3.
4. Stack the pretrained RBMs into a deep hierarchical model.
5. Fine-tune the entire network for the target recommendation or content task.

3.4 Evaluation

Metric	Purpose
Precision@K	Fraction of top-K recommendations that are relevant.
Recall@K	Fraction of relevant items recovered in top-K.
NDCG@K	Rewards correct ranking with higher weight on the top of the list.
MRR	Useful when the first relevant recommendation matters.
Cold-start evaluation	Checks behavior for users/items with limited interaction history.

4. Question 4 - Design a Stochastic and Contractive Representation Model

4.1 Problem definition

A real-world image or signal-processing system must remain useful when the input changes slightly because of sensor noise, illumination, compression, or small shifts. A Contractive Autoencoder (CAE) is proposed because it explicitly penalizes excessive sensitivity of the encoder to small input changes.

4.2 Architecture

Stage	Design
Input	64 x 64 grayscale image, scaled to [0,1], or an equivalent signal window.
Encoder	Convolutional blocks with ReLU, followed by a 32-unit latent layer.
Decoder	Mirrored upsampling / transposed-convolution blocks.
Output	Sigmoid image reconstruction in [0,1].
Robustness mechanism	Jacobian penalty plus small realistic input perturbations during training.

Q4 - Contractive Representation Model

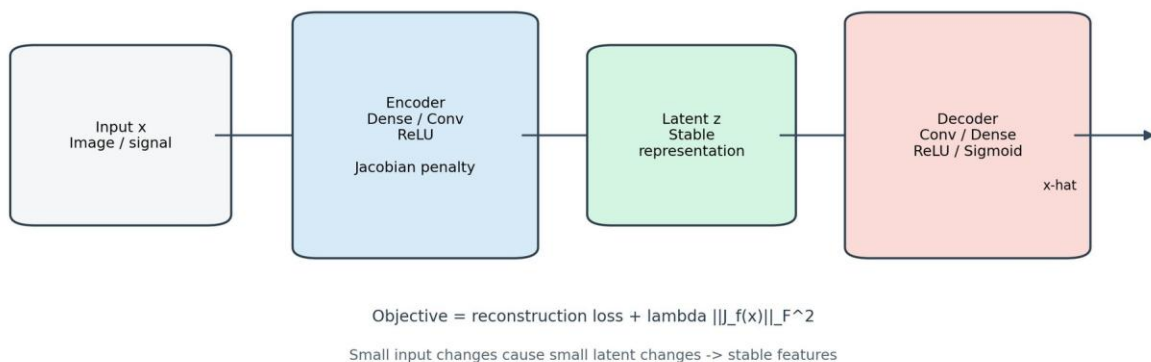


Figure 5. Contractive representation model emphasizing local stability around each input.

4.3 Objective function

Contractive objective: $L_{\text{total}} = L_{\text{rec}} + \lambda_c * ||J_f(x)||_F^2$, where $f(x)$ is the encoder mapping and $J_f(x)$ is its Jacobian with respect to the input. The reconstruction term preserves information; the Jacobian penalty reduces local sensitivity.

4.4 Robustness mechanism

1. Generate a small perturbation δ , such as low-amplitude Gaussian noise or a slight intensity change.
2. Encode both x and $x + \delta$.
3. Encourage the representation to remain stable while still reconstructing useful structure.
4. Use the Jacobian penalty to suppress large local derivatives of the encoder.

4.5 Evaluation

Experiment	Metric	Expected behavior
Clean reconstruction	MSE / MAE	Acceptable baseline reconstruction.
Perturbed reconstruction	MSE / MAE after controlled perturbation	Small degradation relative to clean input.
Latent stability	$ z(x+\delta) - z(x) $	Smaller change for small delta is better.
Experiment	Metric	Expected behavior
Downstream task	Accuracy / F1 under clean and perturbed inputs	Performance gap should be smaller than a non-robust baseline.

5. Question 5 - Design a GAN-Based Data Generation System

5.1 Problem definition

An organization has limited training images and wants additional realistic samples. A Generative Adversarial Network is appropriate because generation is learned directly: a Generator attempts to synthesize realistic images while a Discriminator attempts to distinguish real from generated images.

5.2 Proposed GAN architecture

Component	Proposed design
Generator input	100-dimensional latent vector z sampled from a standard normal distribution.
Generator	Dense projection followed by upsampling / transposed convolution blocks.
Generator output	3-channel image with Tanh activation after scaling images to a matching range.
Discriminator	Convolutional blocks with LeakyReLU followed by a dense probability output.
Discriminator target	1 for real image and 0 for generated image in the classical formulation.

Q5 - GAN-Based Synthetic Image Generation

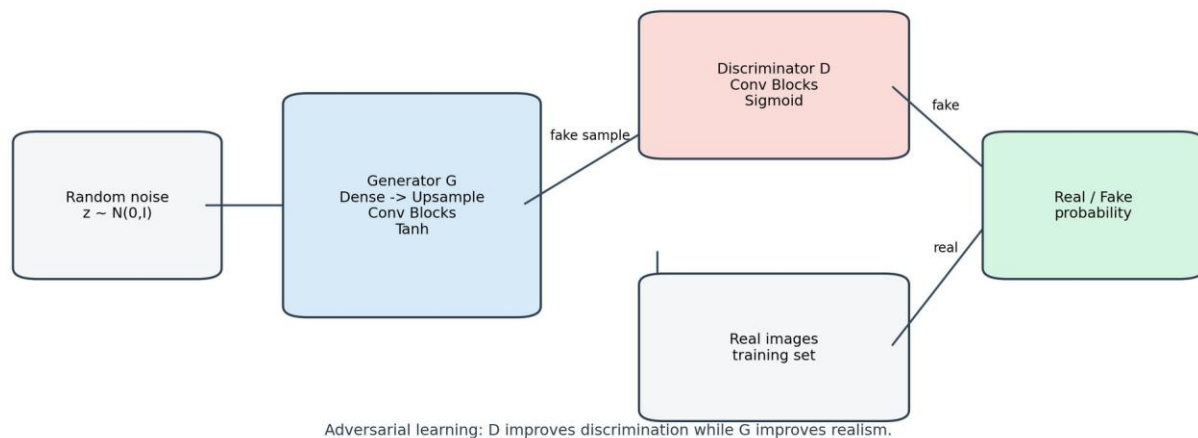


Figure 6. GAN training flow with Generator, Discriminator, real images and generated samples.

5.3 Adversarial learning cycle

1. Sample real images from the training set and latent vectors z from the prior distribution.
2. Generate fake images $G(z)$.
3. Train the Discriminator to distinguish real images from $G(z)$.
4. Train the Generator so that $D(G(z))$ is interpreted as real.
5. Alternate these updates until the generated distribution becomes visually and statistically closer to the training distribution.

Classical minimax objective: $\min_G \max_D E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$. Practical GANs may use a non-saturating generator loss for stronger gradients.

5.4 Evaluation strategy

Measure	Purpose
Visual inspection	Texture, shape, class consistency and obvious artifacts.
FID	Compares feature statistics of real and generated samples; lower is generally better when computed consistently.
Precision / Recall for generative models	Separates fidelity from coverage when implemented.
Nearest-neighbor analysis	Checks whether samples are near-duplicates of training images.
Downstream augmentation test	Checks whether synthetic samples improve the target classifier when added to real training data.

5.5 Why GAN instead of an autoencoder?

Aspect	Autoencoder	GAN
Primary focus	Representation and reconstruction	New sample generation
Typical output behavior	Can become smooth or blurry under pixel-wise reconstruction losses	Can produce sharper, more realistic looking samples
Main strength	Latent representation and reconstruction	Distribution matching / sample realism
Main challenge	May reconstruct but not generate the desired diversity	Training can be unstable and sensitive to Generator/Discriminator balance
Best fit in this problem	Useful for compression or representation	Directly matches the objective of creating additional realistic images